Software Engineering

Paulo Borba
Informatics Center
Federal University of Pernambuco

pauloborba.cin.ufpe.br

Project management

Management goal is to monitor and control…

- Team (communication)
- Productivity (waste)
- Risks
- Scope
- Time
- Quality
- Cost

Team communication

Different kinds of meetings

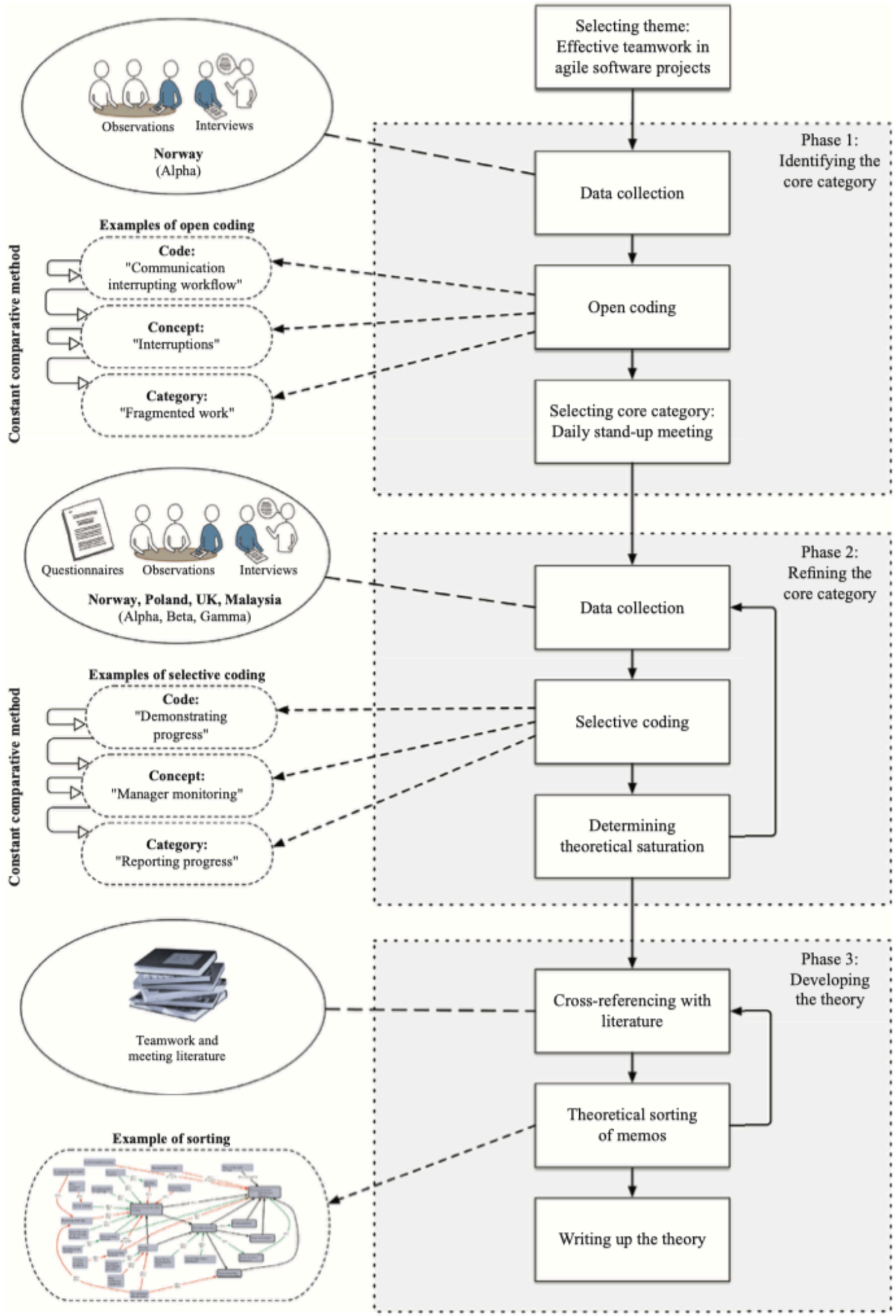
- planning meetings
- review meetings
- retrospective meetings
- validation meetings
- daily (scrum) meetings (15 min)
  - done? to do? obstacles?

What contributes to effective teamwork?

Which are the benefits and drawbacks of DSMs?

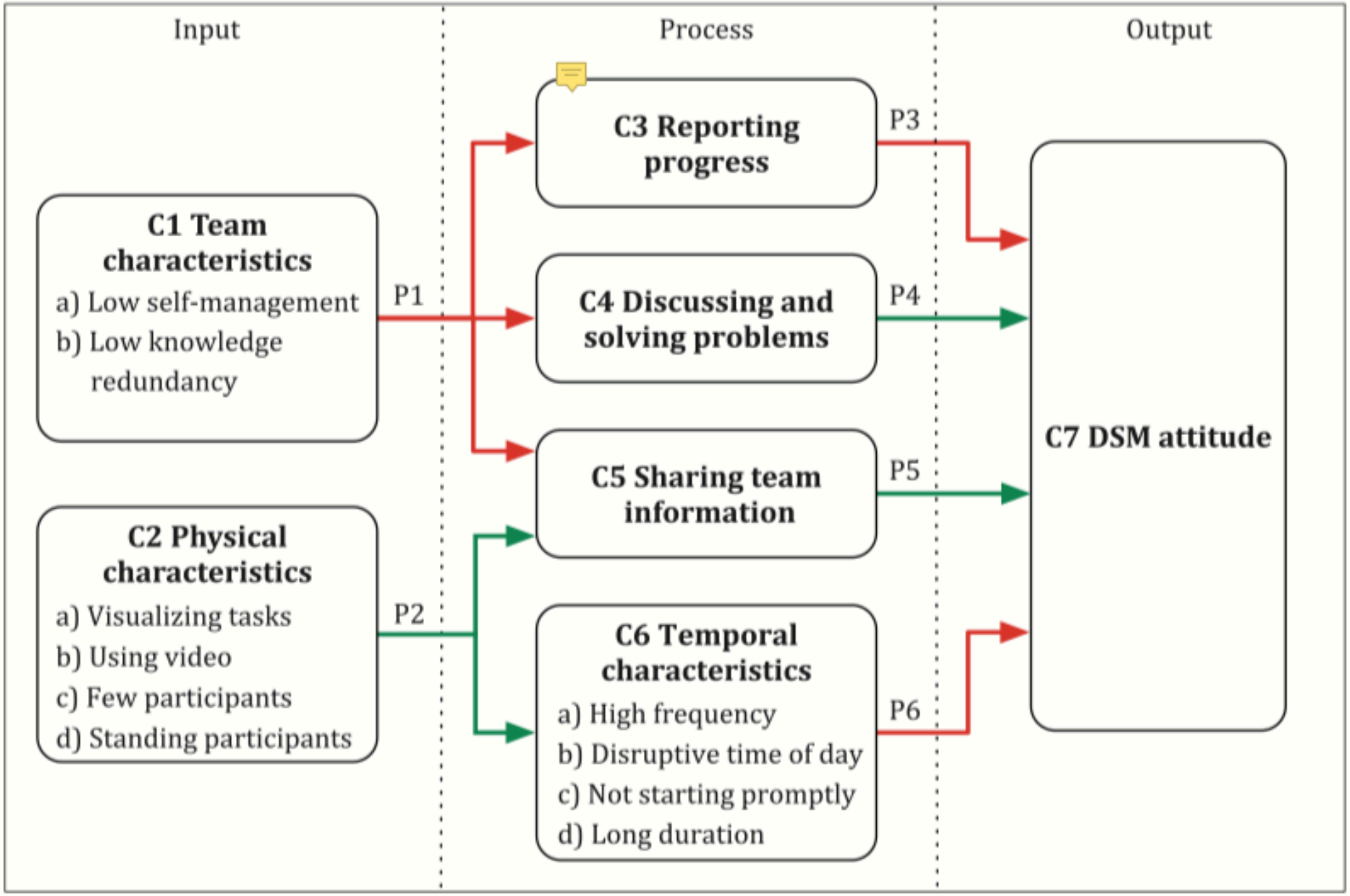
(daily stand-up meetings)

The daily stand-up meeting: A grounded theory study.
Viktoria Stray et al. JSS 2016.


Selecting theme: Effective teamwork in agile software projects
Observations
Interviews
Norway (Alpha)
Phase 1: Identifying the core category
Data collection
Examples of open coding
Code: "Communication interrupting workflow"
Concept: "Interruptions"
Category: "Fragmented work"
Constant comparative method
Open coding
Selecting core category: Daily stand-up meeting
Questionnaires
Observations
Interviews
Norway, Poland, UK, Malaysia (Alpha, Beta, Gamma)
Phase 2: Refining the core category
Data collection
Examples of selective coding
Code: "Demonstrating progress"
Concept: "Manager monitoring"
Category: "Reporting progress"
Constant comparative method
Selective coding
Determining theoretical saturation
Teamwork and meeting literature
Phase 3: Developing the theory
Cross-referencing with literature
Example of sorting
Theoretical sorting of memos
Writing up the theory

Theory

main concepts and relationships


Input
Process
Output
C1 Team characteristics
a) Low self-management
b) Low knowledge redundancy
C2 Physical characteristics
a) Visualizing tasks
b) Using video
c) Few participants
d) Standing participants
P1
P2
C3 Reporting progress
C4 Discussing and solving problems
C5 Sharing team information
C6 Temporal characteristics
a) High frequency
b) Disruptive time of day
c) Not starting promptly
d) Long duration
P3
P4
P5
P6
C7 DSM attitude

DSMs benefits

- information sharing
  - improves collaboration and coordination
  - team awareness
  - avoids conflicts, or reveal them earlier
- opportunity to discuss and solve (very quickly) problems
- better team cohesion, spirit, culture, and training

DSMs drawbacks
(and solutions)

- disruptive, interruption
  - as any meeting, no solution, but
  - find the least disruptive time (just before lunch?)
- delay to start, to finish
  - just develop culture to start on time
  - < 15 min

more DSMs drawbacks
(and solutions)

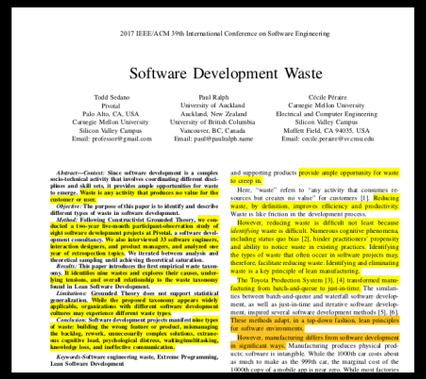
- boring status reporting
  - just drop one of the questions,
  - use tool or board
- too frequent
  - (just reduce frequency)
  - maybe not everyday, but 2-3 days a week
  - more in the middle of the sprint?

DSMs guidelines

- avoid the facilitator allocating turns
  - use a round-robin approach, to avoid having a 1-n meeting instead of a m-n
- team should have cohesive tasks
  - too big group might loose that, and
  - people might loose interest in others tasks

Productivity waste

Waste is unnecessary activity that produces no value for stakeholders



2017 IEEE/ACM 39th International Conference on Software Engineering

Software Development Waste

Todd Sedano
Pivotal
Palo Alto, CA, USA
Carnegie Mellon University
Silicon Valley Campus
Email: professor@gmail.com

Paul Ralph
University of Auckland
Auckland, New Zealand
University of British Columbia
Vancouver, BC, Canada
Email: paul@paulralph.name

Cécile Péraire
Carnegie Mellon University
Electrical and Computer Engineering
Silicon Valley Campus
Moffett Field, CA 94035, USA
Email: cecile.peraire@cmu.edu

Abstract—Context: **Since software development is a complex socio-technical activity that involves coordinating different disciplines and skill sets, it provides ample opportunities for waste to emerge. Waste is any activity that produces no value for the customer or user.**

Objective: **The purpose of this paper is to identify and describe different types of waste in software development.**

Method: **Using Grounded Theory and Constructivist Grounded Theory, we conducted a twelve-year fourteen-month participant-observation study of eight software development projects at Pivotal, a software development consultancy. We also interviewed 33 software engineers, interaction designers, and product managers, and analyzed one year of retrospection topics. We iterated between analysis and theoretical sampling until achieving theoretical saturation.**

Results: **This paper introduces the first empirical waste taxonomy. It identifies nine wastes and explores their causes, underlying tensions, and overall relationship to the waste taxonomy found in Lean Software Development.**

Limitations: **Grounded Theory does not support statistical generalization. While the proposed taxonomy appears widely applicable, organizations with different software development cultures may experience different waste types.**

Conclusion: **Software development projects manifest nine types of waste: building the wrong feature or product, micromanaging the backlog, rework, unnecessarily complex solutions, extraneous cognitive load, psychological distress, waiting/multitasking, knowledge loss, and ineffective communication.**

Keywords-Software engineering waste, Extreme Programming, Lean Software Development

and supporting products provide ample opportunity for waste to creep in.

Here, "waste" refers to "any activity that consumes resources but creates no value" for customers [1]. Reducing waste, by definition, improves efficiency and productivity. Waste is like friction in the development process.

However, reducing waste is difficult not least because *identifying* waste is difficult. Numerous cognitive phenomena, including status quo bias [2], hinder practitioners' perspicuity and ability to notice waste in existing practices. Identifying the types of waste that often occur in software projects may, therefore, facilitate reducing waste. Identifying and eliminating waste is a key principle of lean manufacturing.

The Toyota Production System [3], [4] transformed manufacturing from batch-and-queue to just-in-time. The similarities between batch-and-queue and waterfall software development, as well as just-in-time and iterative software development, inspired several software development methods [5], [6]. These methods adapt, in a top-down fashion, lean principles for software environments.

However, manufacturing differs from software development in significant ways. Manufacturing produces physical products; software is intangible. While the 1000th car costs about as much to make as the 999th car, the marginal cost of the 1000th copy of a mobile app is near zero. While most factories

Software Development Waste.
Todd Sedano et al. ICSE 2017.

Reducing waste
improves productivity,
by definition

What are the most common waste categories and their associated causes?

Data collection

(at Pivotal Labs)

- Engineer field notes in 8 projects
- 33 interviews (interaction designers, product managers, and software engineers)
- 91 retrospective meetings (over 59 weeks, 3 projects)

focus on answering the previous question

- Waste Category: Psychological Distress
  - Cause Property: Low team morale
    - Retro Topic: Frustrated developers
    - Retro Topic: Not managing expectations
    - Retro Topic: Negative attitudes
    - Retro Topic: Apathy
    - Retro Topic: Not knowing everyone on the team
    - Retro Topic: Project feels like it is falling apart emotionally
    - Retro Topic: Unacknowledged by management
    - Retro Topic: Messy code decreasing sense of ownership
  - Cause Property: Rush mode
    - Retro Topic: Fixed set of features with a fixed timeline
    - Retro Topic: Aggressive timelines
    - Retro Topic: Shifting deadline
    - Retro Topic: Scope creep
    - Retro Topic: Repeatedly hearing “This is due today”
    - Retro Topic: Long days
    - Retro Topic: Overtime
  - Cause Property: Interpersonal or team conflict
    - Retro Topic: Criticizing in public
    - Retro Topic: Difficult pairings
    - Retro Topic: Pairing fatigue
    - Retro Topic: Not listening
    - Retro Topic: Interpersonal conflict

Waste	Description	Observed Causes
Building the wrong feature or product	The cost of building a feature or product that does not address user or business needs.	User desiderata (not doing user research, validation, or testing; ignoring user feedback; working on low user value features) Business desiderata (not involving a business stakeholder; slow stakeholder feedback; unclear product priorities)
Mismanaging the backlog	The cost of duplicating work, expediting lower value user features, or delaying necessary bug fixes.	Backlog inversion Working on too many features simultaneously Duplicated work Not enough ready stories Imbalance of feature work and bug fixing Delaying testing or critical bug fixing Capricious thrashing
Rework	The cost of altering delivered work that should have been done correctly but was not.	Technical debt Rejected stories (e.g. product manager rejects story implementation) No clear definition of done (ambiguous stories; second guessing design mocks) Defects (poor testing strategy; no root-cause analysis on bugs)
Unnecessarily complex solutions	The cost of creating a more complicated solution than necessary, a missed opportunity to simplify features, user interface, or code.	Unnecessary feature complexity from the user's perspective Unnecessary technical complexity (duplicating code, lack of interaction design reuse, overly complex technical design created up-front)
Extraneous cognitive load	The costs of unneeded expenditure of mental energy.	Suffering from technical debt Complex or large stories Inefficient tools and problematic APIs, libraries, and frameworks Unnecessary context switching Inefficient development flow Poorly organized code
Psychological distress	The costs of burdening the team with unhelpful stress.	Low team morale Rush mode Interpersonal or team conflict
Waiting/multitasking	The cost of idle time, often hidden by multi-tasking.	Slow tests or unreliable tests Unreliable acceptance environment Missing information, people, or equipment Context switching from delayed feedback
Knowledge loss	The cost of re-acquiring information that the team once knew.	Team churn Knowledge silos
Ineffective communication	The cost of incomplete, incorrect, misleading, inefficient, or absent communication.	Team size is too large Asynchronous communication (distributed teams; distributed stakeholders; dependency on another team; opaque processes outside team) Imbalance (dominating the conversation; not listening) Inefficient meetings (lack of focus; skipping retros; not discussing blockers each day; meetings running over (e.g. long stand-ups))

Which listed waste category is more important for you?

Which listed waste category is less important for you?

Are you aware of a waste category not listed here? Which one?

Waste	Description
Building the wrong feature or product	The cost of building a feature or product that does not address user or business needs.
Mismanaging the backlog	The cost of duplicating work, expediting lower value user features, or delaying necessary bug fixes.
Rework	The cost of altering delivered work that should have been done correctly but was not.
Unnecessarily complex solutions	The cost of creating a more complicated solution than necessary, a missed opportunity to simplify features, user interface, or code.
Extraneous cognitive load	The costs of unneeded expenditure of mental energy.
Psychological distress	The costs of burdening the team with unhelpful stress.
Waiting/multitasking	The cost of idle time, often hidden by multi-tasking.
Knowledge loss	The cost of re-acquiring information that the team once knew.
Ineffective communication	The cost of incomplete, incorrect, misleading, inefficient, or absent communication.

Which are the most critical waste causes for you?

During the explanation of the waste categories, keep track of the most critical waste causes for you.

Building wrong feature/product
(cost of creating software that does not address needs)

Affects team morale and customer satisfaction

Caused by tension between user needs and business wants, persona vs real users

- user desiderata (not doing user research, validation, or testing; ignoring user feedback; working on low user value features)
- business desiderata (not involving a business stakeholder; slow stakeholder feedback; unclear product priorities)

Mismanaging the backlog

(cost of duplicating work, or delaying relevant features and bug fixes)

Causes delays and duplicate work

Caused by unclear product priorities and tensions

- working on low priority tasks (backlog inversion)
- duplicate stories, engineers addressing the same task
- starting work, changing priorities, and halting
- working on too many features simultaneously
- not enough ready stories
- capricious adjustments

Rework

(cost of altering delivered work)

Causes delays, and might affect team morale and customer satisfaction

Caused by

- technical debt
- defects in code-stories-tests-mockups (poor testing strategy; no root-cause analysis on bugs)
- rejected stories
- stories with no clear acceptance criteria

Unnecessarily complex solutions

(cost of creating a more complicated solution than necessary)

Causes harder maintenance

Caused by tension between big design up-front versus incremental design

- no amount of up-front consideration appears sufficient to predict user feedback
- extra feature complexity from the user's perspective
- extra design and implementation complexity

Extraneous cognitive load
(cost of unneeded expenditure of mental energy)

Caused by

- suffering from technical debt
- complex or large stories
- inefficient tools and problematic APIs, libraries, and frameworks
- unnecessary context switching
- inefficient development flow

Psychological distress
(cost of burdening the team with unhelpful stress)

Caused by

- low team morale
- rush mode
- interpersonal conflicts
- *unsolved (retrospective, assessment) complaints*
- *lack of perspective on the job*

Waiting/multitasking

(cost of idle time, often hidden by multi-tasking)

Caused by tension between wait, block or guess, when needed information is not available

- slow or unreliable tests (*or builds*)
- unreliable acceptance environment
- missing information, people, or equipment
- context switching from delayed feedback
- working on low priority tasks for avoiding idle time

Knowledge loss

(cost of re-acquiring information that the team once knew)

Caused by

- team churn (members leaving the team)
- knowledge silos

Ineffective communication

(cost of incomplete, incorrect, misleading, inefficient, or absent communication)

Caused by

- team size is too large
- asynchronous communication (distributed teams; distributed stakeholders; dependency on another team; opaque processes outside team)
- imbalance (dominating the conversation; not listening)
- inefficient meetings (lack of focus; skipping retros; not discussing blockers each day; long meetings)

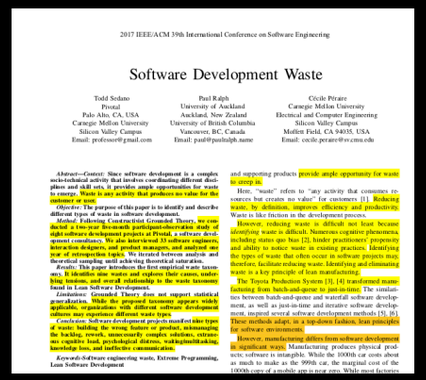
Which are the most critical waste causes for you?

Quality and scope

Time and cost

Fixing both scope and schedule is often antithetical for systems that are not similar to previously developed systems.

"The client either chooses the release date and gets only the features ready by then or chooses key features and ships the product when the features are ready."



2017 IEEE/ACM 39th International Conference on Software Engineering

Software Development Waste

Todd Sedano
Pivotal
Palo Alto, CA, USA
Carnegie Mellon University
Silicon Valley Campus
Email: professor@gmail.com

Paul Ralph
University of Auckland
Auckland, New Zealand
University of British Columbia
Vancouver, BC, Canada
Email: paul@paulralph.name

Cécile Péraire
Carnegie Mellon University
Electrical and Computer Engineering
Silicon Valley Campus
Moffett Field, CA 94035, USA
Email: cecile.peraire@ece.cmu.edu

Abstract—*Context:* **Since software development is a complex socio-technical activity that involves coordinating different disciplines and skill sets, it provides ample opportunities for waste to emerge. Waste is any activity that produces no value for the customer or user.**

Objective: **The purpose of this paper is to identify and describe different types of waste in software development.**

Method: **Following Constructivist Grounded Theory, we conducted a 14-to-year 16-month participant-observation study of eight software development projects at Pivotal, a software development consultancy. We also interviewed 33 software engineers, interaction designers, and product managers, and analyzed one year of retrospection logs. We iterated between analysis and theoretical sampling until achieving theoretical saturation.**

Results: **We identify 17 types of waste in software development, develop a waste taxonomy, describe each waste and explore their causes, underlying tensions, and overall relationship to the waste taxonomy found in Lean Software Development.**

Limitations: **Grounded Theory does not support statistical generalization. While the proposed taxonomy appears widely applicable, organizations with different software development cultures may experience different waste types.**

Conclusion: **Software development projects manifest nine types of waste: building the wrong feature or product, accumulating the backlog, rework, unnecessarily complex solutions, extraneous cognitive load, psychological distress, waiting/multitasking, knowledge loss, and ineffective communication.**

Keywords-Software engineering waste, Extreme Programming, Lean Software Development

and supporting products provide ample opportunity for waste to creep in.

Here, "waste" refers to "any activity that consumes resources but creates no value" for customers [1]. Reducing waste, by definition, improves efficiency and productivity. Waste is like friction in the development process.

However, reducing waste is difficult not least because identifying waste is difficult. Numerous cognitive phenomena, including status quo bias [2], hinder practitioners' propensity and ability to notice waste in existing practices. Identifying the types of waste that often occur in software projects may, therefore, facilitate reducing waste. Identifying and eliminating waste is a key principle of lean manufacturing.

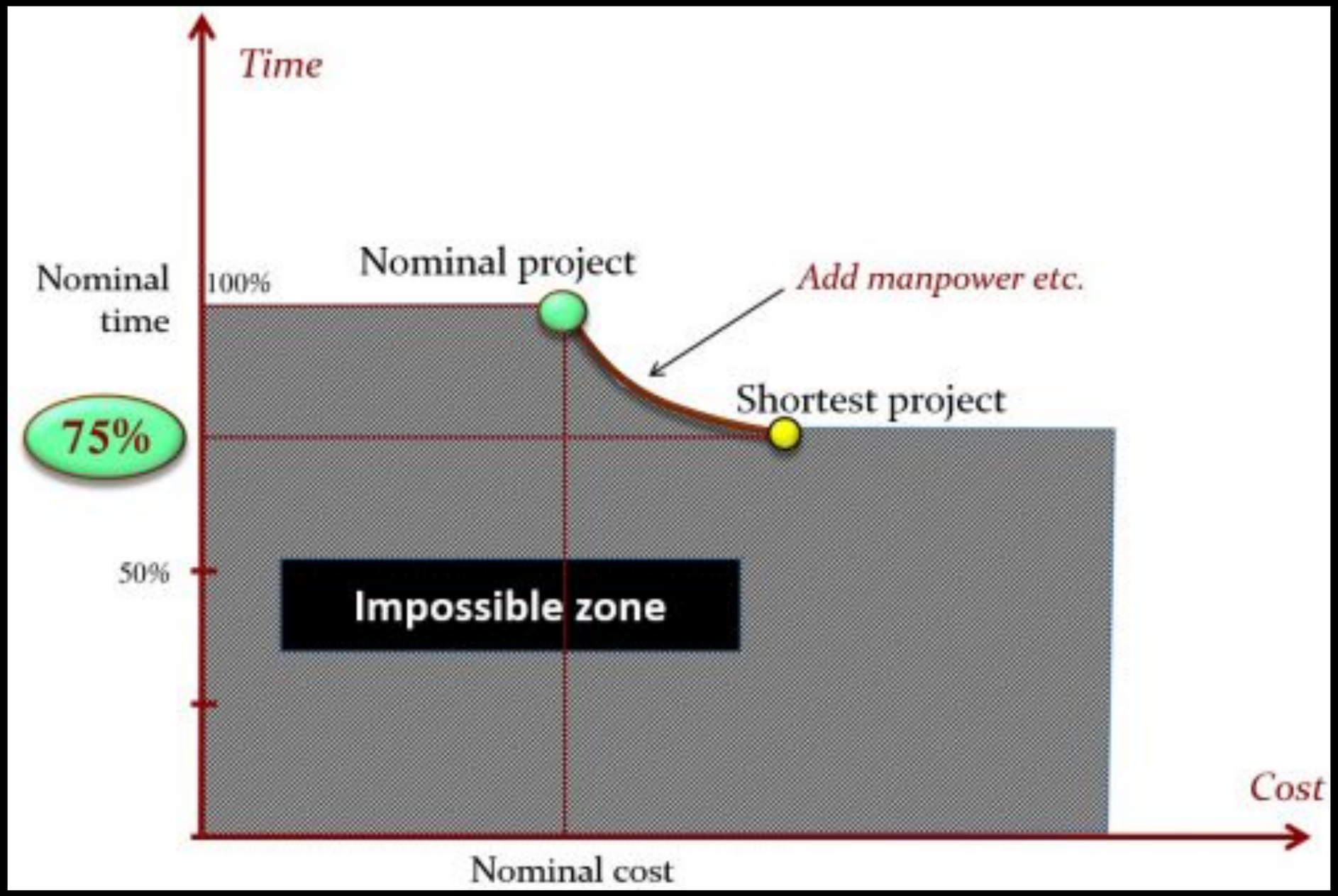
The Toyota Production System [3], [4] transformed manufacturing from batch-and-queue to just-in-time. The similarities between batch-and-queue and waterfall software development, as well as just-in-time and iterative software development, inspired several software development methods [5], [6]. These methods adapt, in a top-down fashion, lean principles for software environments.

However, manufacturing differs from software development in significant ways. Manufacturing produces physical products; software is intangible. While the 1000th car costs about as much to make as the 999th car, the marginal cost of the 1000th copy of a mobile app is near zero. While most features

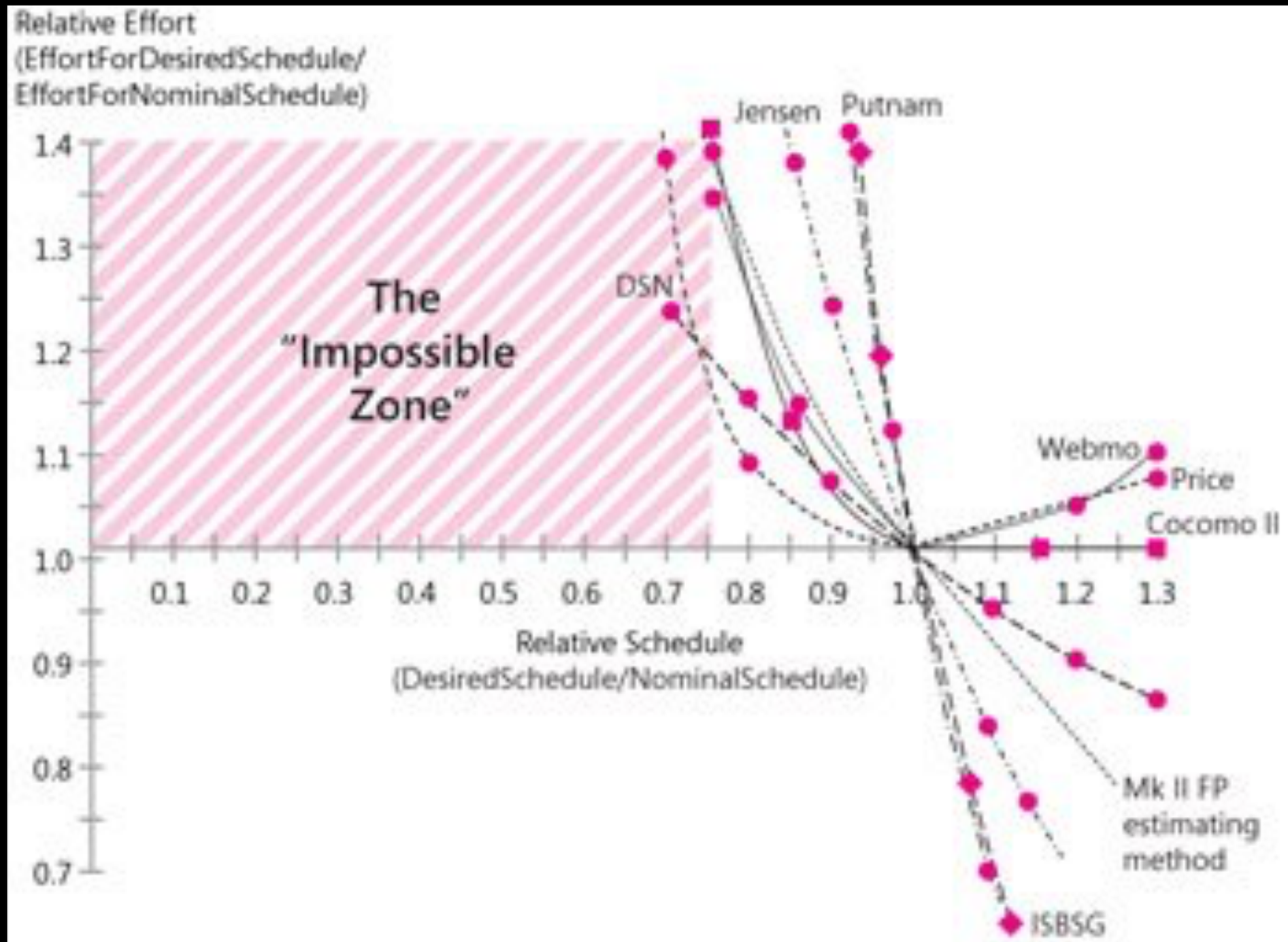
Software Development Waste.
Todd Sedano et al. ICSE 2017.

Brooks' Law: adding people to a late project delays it further

The Shortest Possible Schedule law



https://bertrandmeyer.com/2019/11/01/theorem-software-engineering/



Source: Adapted and extended from Software Sizing and Estimating: Mk II *(Symons 1991),* Software Cost Estimation with Cocomo II *(Boehm et al 2000), "Estimating Web Development Costs: There Are Differences" (Reifer 2002), and* Practical Project Estimation, 2nd Edition *(ISBSG 2005).*

You can, within limits, shorten delivery times by properly bringing people to the project

Software Engineering

Paulo Borba
Informatics Center
Federal University of Pernambuco

pauloborba.cin.ufpe.br